

+++ date = '2026-02-13T17:58:46-08:00' draft = false title = 'Práctica 3: El paradigma funcional' +++

1. Introducción

1.1 Objetivo

El objetivo de esta práctica fue instalar y configurar el entorno de desarrollo de Haskell, explorar las herramientas del ecosistema y analizar una aplicación real escrita en este lenguaje: una aplicación TODO de línea de comandos. A diferencia de la práctica anterior —centrada en el paradigma orientado a objetos con Python y Django—, esta práctica introduce el **paradigma funcional**, donde las funciones puras, la inmutabilidad y el sistema de tipos son los pilares del diseño.

1.2 Herramientas del ecosistema Haskell

La instalación se realizó mediante **GHCCup**, el instalador oficial de Haskell, ejecutando el siguiente comando en PowerShell (sin modo administrador):

```
Set-ExecutionPolicy Bypass -Scope Process -Force;  
[System.Net.WebClient]::new().DownloadString('https://www.haskell.org/ghcup/sh/boo  
tstrap-haskell.ps1') | Invoke-Expression
```

Este comando instala automáticamente todos los componentes del ecosistema:

Herramienta	Versión	Función
GHCCup	0.1.50.2	Instalador y gestor del entorno Haskell
GHC	9.6.7	Compilador oficial de Haskell (Glasgow Haskell Compiler)
GHCi	9.6.7	Intérprete interactivo (REPL) de Haskell
HLS	Incluido	Haskell Language Server — provee librerías estándar y soporte a IDEs
Stack	Incluido	Manejador de paquetes, similar a pip en Python
Cabal	3.14.2.0	Herramienta de empaquetado y build; usa Stack y GHC en conjunto

1.3 Verificación de la instalación

Se confirmó la correcta instalación ejecutando en PowerShell:

```
PS C:\Users\tranz> ghc --version  
The Glorious Glasgow Haskell Compilation System, version 9.6.7  
  
PS C:\Users\tranz> cabal --version  
cabal-install version 3.14.2.0  
compiled using version 3.14.2.0 of the Cabal library
```

```
PS C:\Users\tranz> ghcup --version
The GHCup Haskell installer, version 0.1.50.2
```

Adicionalmente, se instaló la **extensión oficial de Haskell para Visual Studio Code**, que se conecta con HLS para proveer resaltado de sintaxis, autocompletado y diagnósticos en tiempo real.

2. Desarrollo de la práctica

2.1 La aplicación TODO

La aplicación analizada es un gestor de tareas interactivo en consola escrito en Haskell. A diferencia de otras implementaciones basadas en argumentos de línea de comandos, esta aplicación funciona mediante comandos ingresados dinámicamente por el usuario.

Los comandos disponibles son:

- `+ <String>` — Agrega una nueva tarea
- `- <Int>` — Elimina la tarea indicada
- `s <Int>` — Muestra una tarea específica
- `e <Int>` — Edita una tarea
- `l` — Lista todas las tareas
- `r` — Invierte el orden de la lista
- `c` — Limpia todas las tareas
- `q` — Sale del programa

2.2 Estructura del proyecto

Al clonar el repositorio y acceder al directorio de la aplicación, se encontró la siguiente estructura de archivos:

```
todo/
├── todo.cabal      ← Configuración del proyecto: dependencias y punto de
entrada
├── src/
│   ├── Main.hs    ← Código fuente principal de la aplicación
└── README.md
```

El archivo `.cabal` define el ejecutable, las dependencias externas y el módulo principal. Los archivos de código fuente en Haskell utilizan la extensión `.hs`.

2.3 Análisis del código fuente

Módulos e importaciones

```
module Main where

import Configuration.Dotenv (defaultConfig, loadFile)
```

```
import Lib (prompt)
import System.Environment (lookupEnv)
import Web.Browser (openBrowser)
```

Haskell organiza el código en módulos, lo que permite una gestión clara de las dependencias. En este caso, se importan únicamente las funciones necesarias para la ejecución del programa. La librería `Configuration.Dotenv` se encarga de cargar variables de entorno desde un archivo `.env`, `System.Environment` permite acceder a dichas variables en el sistema, mientras que `Web.Browser` facilita la interacción con el navegador predeterminado. Por último, el módulo `Lib` centraliza la lógica de negocio y el núcleo interactivo de la aplicación.

Tipo de datos para las tareas

```
data Task = Task
  { taskId    :: Int,
    taskDesc  :: String,
    taskDone  :: Bool
  } deriving (Show, Read)
```

`Task` es un *tipo de dato algebraico (ADT)* definido mediante la sintaxis de registros (records). La cláusula `deriving (Show, Read)` es fundamental aquí: le indica al compilador que genere automáticamente las funciones para convertir el tipo de dato a texto (`show`) y para reconstruirlo a partir de una cadena de texto (`read`). Esto permite implementar la persistencia de datos en archivos de texto plano de manera sumamente sencilla, sin necesidad de recurrir a bases de datos complejas.

La mónada IO y los efectos secundarios

```
main :: IO ()
main = do
  loadFile defaultConfig
  website <- lookupEnv "WEBSITE"
```

La función `main` tiene el tipo `IO ()`, lo que señala explícitamente que el programa interactúa con el mundo exterior. En Haskell, todas las operaciones que conllevan efectos secundarios (como leer archivos con `loadFile` o consultar el entorno con `lookupEnv`) deben encapsularse en la mónada IO. Esta distinción es clave en el paradigma funcional, ya que permite aislar el código puro (predecible y fácil de testear) de las interacciones con el sistema, mejorando la robustez del software.

Pattern matching en el despachador de comandos

```
case website of
  Nothing -> error "You should set WEBSITE at .env file."
```

```
Just s -> do
  result <- openBrowser s
```

El *pattern matching* (ajuste de patrones) sobre el tipo **Maybe** (que puede ser **Just** con un valor o **Nothing**) es la forma segura en que Haskell maneja la ausencia de datos. A diferencia de otros lenguajes donde un valor nulo podría causar un error en tiempo de ejecución, aquí el compilador obliga al programador a contemplar ambos casos. Este mecanismo garantiza que el flujo del programa sea exhaustivo y reduce drásticamente los errores por valores no definidos.

Funciones de alto orden para manipular listas

```
markDone :: Int -> IO ()
markDone n = do
  tasks <- loadTasks
  let updated = map (\t -> if taskId t == n
                          then t { taskDone = True }
                          else t) tasks
  saveTasks updated
```

En este bloque se observa el uso de **map**, una función de alto orden que aplica una lógica específica a cada elemento de una lista. Es un ejemplo perfecto de la inmutabilidad en Haskell: la función no modifica la lista **tasks** original, sino que genera una nueva versión llamada **updated** con los cambios aplicados. Esta ausencia de "efectos colaterales" en los datos previene errores comunes de concurrencia y estado compartido, facilitando el mantenimiento del código a largo plazo.

2.4 Intento de ejecución y problemas encontrados

Se intentó compilar y ejecutar la aplicación con los comandos estándar de Cabal:

```
cabal update
cabal build
cabal run
```

Durante el proceso se presentaron **problemas de compilación** debidos a dependencias externas y a la configuración del entorno en Windows. Específicamente, algunas librerías del proyecto requerían herramientas del sistema (como **mingw**) que no estaban completamente configuradas en la ruta del sistema, lo que interrumpió el proceso de build.

A pesar de no poder ejecutar la aplicación, se realizó un **análisis estático completo del código fuente** en VSCode, lo que permitió comprender el funcionamiento de la aplicación y los conceptos del paradigma funcional aplicados en ella.

2.5 Conceptos del paradigma funcional identificados

Concepto	Aplicación en el proyecto
Funciones puras	Las funciones de transformación de listas (<code>map</code> , <code>filter</code>) no producen efectos secundarios.
Tipos de datos algebraicos	<code>Task</code> modela el dominio sin clases ni herencia.
Pattern matching	El despachador de comandos en <code>main</code> cubre todos los casos posibles.
Mónada IO	Aísla los efectos secundarios (I/O de archivos, argumentos) del código puro.
Inmutabilidad	<code>markDone</code> construye una nueva lista en lugar de modificar la existente.
Funciones de orden superior	<code>map</code> recibe una función como argumento para transformar la lista de tareas.
Sistema de tipos estático	El compilador rechaza usos incorrectos de tipos antes de ejecutar el programa.

3. Conclusiones

Esta práctica evidenció las diferencias fundamentales entre el paradigma orientado a objetos —explorado en la práctica anterior— y el paradigma funcional de Haskell.

En POO, el estado se distribuye en objetos que se modifican a lo largo del tiempo. En Haskell, el estado es explícito e inmutable: las transformaciones producen nuevos valores en lugar de mutar los existentes. Esta distinción hace que el código funcional sea más fácil de razonar, ya que una función siempre produce el mismo resultado para los mismos argumentos, independientemente del orden de ejecución o del estado del sistema.

El sistema de tipos de Haskell resultó ser una herramienta de diseño, no solo de verificación. Al declarar `Task` como un tipo de dato algebraico, el compilador garantiza que ninguna parte del programa puede crear una tarea con campos faltantes o de tipo incorrecto. El uso de `deriving (Show, Read)` ilustra cómo el compilador puede generar código repetitivo automáticamente, reduciendo errores.

La mónada IO, aunque inicialmente contraintuitiva, tiene un propósito claro: separar el código puro —sin efectos secundarios, fácilmente testeable— del código que interactúa con el mundo exterior. Esta separación forzada por el tipo es una de las características más poderosas y características de Haskell.

Como área de oportunidad, los problemas de configuración del entorno en Windows resaltan que el ecosistema Haskell está más maduro en sistemas Unix. Para proyectos futuros, utilizar WSL (Windows Subsystem for Linux) o un contenedor Docker simplificaría la instalación de dependencias del sistema.

4. Referencias

1. Hutton, G. (2016). *Programming in Haskell* (2nd ed.). Cambridge University Press.
2. O'Sullivan, B., Goerzen, J., & Stewart, D. (2008). *Real World Haskell*. O'Reilly Media.
<http://book.realworldhaskell.org/>
3. Haskell Wiki. (2024). *Haskell/do notation*. https://en.wikibooks.org/wiki/Haskell/do_notation

4. GHCup. (2024). *GHCup — The Haskell toolchain installer*. <https://www.haskell.org/ghcup/>
5. Haskell.org. (2024). *Get Started with Haskell*. <https://www.haskell.org/get-started/>
6. Daumé III, H. (2006). *Yet Another Haskell Tutorial*. <https://www.cs.utah.edu/~hal/docs/daume02yaht.pdf>
7. Chalmers University CSE. (2024). *A Tour of the Haskell Syntax*.
<https://www.cse.chalmers.se/edu/course/TDA555/tourofprelude.html>

Enlaces

[GitHub](#) [GitHub Pages](#)